TD5 & TP5 : Agence DDL

1 Schéma des relations

Le schéma de la base de données est donné ci-dessous (les identifiants sont soulignés) :

LesVilles (nomV, pays)
/ (n, p) ∈ LesVilles ⟺ la ville dont le nom est n, est située dans le pays p. Le nom de la ville est un identifiant (clef de la relation). */*
LesMonuments (nomM, nomV, prix)
/ (nm, nv, p) ∈ LesMonuments ⟺ le monument de nom nm est situé dans la ville nv. Son prix de visite est p euros. */*
LesCircuits (numC, vDep, vArr, prix)
/ (n, vd, va, pr) ∈ LesCircuits ⟺ le circuit touristique identifié par le numéro n, part de la ville vd et se termine dans la ville va. Son prix est de pr, qui ne prend pas en compte le prix des monuments visités. La ville de départ représente le point de rendez-vous avec les accompagnateurs. */*
LesEtapes (numC, rang, vEtape, nbJours)
/ (n, r, ve, nbj) ∈ LesEtapes ⟺ la r^e étape du circuit n se déroule dans la ville ve, où le séjour est de nbj jours. On fait comme hypothèse que lorsqu'une ville est dans un circuit, tous ses monuments sont visités. $r \geq 1$. Les villes de départ et d'arrivée (vArr et vDép de LesCircuits) sont dans LesEtapes lorsqu'elles sont visitées. */*
LesProgrammations (numC, dateDep, nbPlaces)
/ (n, d, nbl) ∈ LesProgrammations ⟺ le circuit identifié par le numéro n, programmé à la date d est offert avec nbl places. Le même circuit peut être programmé à différentes dates. */*
LesReservations (numR, nomC, numC, dateDep, nbRes)
/ (nr, no, nc, d, nbr) ∈ LesRéservations ⟺ le client de nom no, a effectué une réservation identifiée par nr, sur le circuit nc programmé à la date d. Il a réservé nbr places. */*

Les domaines associés sont :

domaine (nomC) = {'Bonemine', 'Corto', etc.}
domaine (vEtape) = domaine (vDep) = domaine (vArr)
= domaine (nomV) = {'Paris', 'Florence', 'Briançon', etc.}
domaine (pays) = {'Italie', 'Finlande', 'France', etc.}
domaine (numC) = domaine (numR) = domaine (nbRes) = domaine (rang)
= domaine (nbJours) = domaine (prix) = domaine (nbPlaces) = entiers > 0
domaine (dateDep) = instant (unité jour)

Les contraintes d'intégrité référentielle sont :

LesReservations[numC, dateDep] ⊆ LesProgrammations[numC, dateDep]
LesProgrammations[numC] = LesCircuits[numC]
LesEtapes[vEtape] ⊆ LesVilles[nomV]
LesCircuits[vArr] ⊆ LesVilles[nomV]
LesEtapes[numC] ⊆ LesCircuits[numC]
LesCircuits[vDep] ⊆ LesVilles[nomV]
LesMonuments[nomV] ⊆ LesVilles[nomV]

Remarques :
- On dit qu'un circuit passe par une ville v, lorsque v est une de ses étapes, ou/et la ville arrivée, ou/et la ville de départ ;
- On dit qu'un circuit visite une ville lorsque celle-ci est une étape de ce circuit.
- On généralise ces définitions aux pays : un circuit passe dans un pays p lorsque l'une de ses étapes, et/ou sa ville arrivée, ou/et sa ville de départ sont situées dans p ; un circuit visite un pays lorsque celui-ci a une fois au moins une étape dans ce pays.

La définition, dans le langage SQL supporté par SQLite, des relations LesVilles, LesMonuments, et LesCircuits, est donnée ci-dessous :

```sql
CREATE TABLE LesVilles (
   nomV TEXT NOT NULL,
   pays TEXT NOT NULL,
   CONSTRAINT pk_vil_nomv PRIMARY KEY (nomV)
);

CREATE TABLE LesMonuments (
   nomM TEXT NOT NULL,
   nomV TEXT NOT NULL,
   prix REAL NOT NULL,
   CONSTRAINT pk_mon_nomm_nomv PRIMARY KEY (nomM, nomV),
   CONSTRAINT fk_mon_nomv FOREIGN KEY (nomV) REFERENCES LesVilles (nomV),
   CONSTRAINT ck_mon_prix CHECK (prix >= 0)
);

CREATE TABLE LesCircuits (
   -- En Oracle une PK est NOT NULL par défaut. Ce n'est pas le cas en SQLite
   numC INTEGER NOT NULL,
    vDep TEXT NOT NULL,
   vArr TEXT NOT NULL,
   prix REAL NOT NULL,
   CONSTRAINT pk_cir_numc PRIMARY KEY (numC),
   CONSTRAINT fk_cir_vdep FOREIGN KEY (vDep) REFERENCES LesVilles (nomV),
   CONSTRAINT fk_cir_varr FOREIGN KEY (vArr) REFERENCES LesVilles (nomV),
   CONSTRAINT ck_cir_prix CHECK (prix > 0),
   CONSTRAINT ck_cir_numc CHECK (numC > 0)
);
```

Question 1 :
Donner dans le langage SQL permettant la création des relations LesEtapes, LesProgrammations et LesReservations.

```sql
CREATE TABLE LesEtapes (
   numC INTEGER NOT NULL,
   rang INTEGER NOT NULL,
   vEtape TEXT NOT NULL,
   nbJours INTEGER NOT NULL,
   CONSTRAINT pk_etp_numc_rang PRIMARY KEY(numC, rang),
   CONSTRAINT fk_etp_numc FOREIGN KEY (numC) REFERENCES LesCircuits (numC),
   CONSTRAINT fk_etp_vetape FOREIGN KEY (vEtape) REFERENCES LesVilles (nomV),
   CONSTRAINT ck_etp_nbjours CHECK (nbJours > 0),
      CONSTRAINT ck_etp_rang CHECK (rang > 0)
);

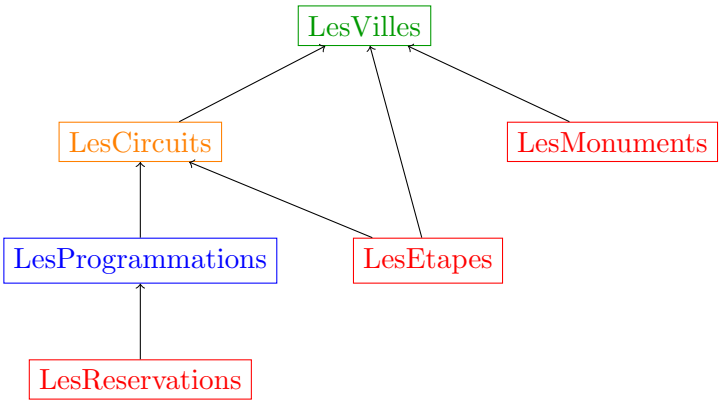
CREATE TABLE LesReservations(
   numR INTEGER NOT NULL,
   nomC TEXT NOT NULL,
   numC INTEGER NOT NULL,
   dateDep DATE NOT NULL,
   nbRes INTEGER NOT NULL,
   CONSTRAINT pk_res_numr PRIMARY KEY (numR),
   CONSTRAINT fk_res_numc_dated FOREIGN KEY (numC, dateDep)
      REFERENCES LesProgrammations (numC, dateDep),
   CONSTRAINT ck_res_nbRes CHECK (nbRes > 0)
);
```

```
CREATE TABLE LesProgrammations (
   numC INTEGER NOT NULL,
   dateDep DATE NOT NULL, -- Différents formats et opérations en Oracle et SQLite
   nbPlaces INTEGER NOT NULL,
CONSTRAINT pk_prog_numc_datedep PRIMARY KEY (numC, dateDep),
CONSTRAINT fk_prog_numc FOREIGN KEY (numC) REFERENCES LesCircuits (numC),
CONSTRAINT ck_prog_nbplaces CHECK (nbPlaces > 0)
);

);
```

Question 2 :
Donner dans le langage SQL, le code permettant la destruction de toutes les relations de la base de données (instruction DROP). **Indication :** attention aux dépendances entre les relations.

Pour trouver l'ordre de suppression, on analyse les dépendances entre les relations (données par les contraintes d'intégrié référentielles). On obtient le graphe de dépendances suivant :



Les relations pouvant être supprimées sont les feuilles dans cet arbre de dépendances : les relations dont aucune autre relation ne dépend, via une contrainte d'intégrité. On supprime ces relations "sans dépendances" jusqu'à ce que toutes les relations aient été supprimées.

```
DROP TABLE IF EXISTS LesReservations;
DROP TABLE IF EXISTS LesProgrammations;
DROP TABLE IF EXISTS LesCircuits;
DROP TABLE IF EXISTS LesEtapes;
DROP TABLE IF EXISTS LesMonuments;
DROP TABLE IF EXISTS LesVilles;
```

Question 3 :
Donner dans le langage SQL, les insertions permettant de définir le circuit 403 avec un prix de base de 1160€, ayant Paris comme ville de départ et d'arrivée. On prévoit d'abord la visite de Paris (3 jours) puis Londres (2 jours). Le circuit est programmé le 04 Mars 2025 (avec 30 places) et le 15 Juillet 2025 (avec 42 places).

Format par défaut Date Oracle : DD/MM/YY (ex. 03/04/25).
Format par défaut Date SQLite : YYYY-MM-DD (ex. 2025-04-03).

```
-- ORACLE
```

```sql
INSERT INTO LesVilles VALUES ('Paris','France');
INSERT INTO LesVilles VALUES ('Londres','Angleterre');

INSERT INTO LesCircuits VALUES (403,'Paris','Paris',1160);

INSERT INTO LesEtapes VALUES (403,1,'Paris', 3);
INSERT INTO LesEtapes VALUES (403,2,'Londres', 2);

INSERT INTO LesProgrammations VALUES (403, TO_DATE ('04-03-2025','DD-MM-YYYY'),30);
INSERT INTO LesProgrammations VALUES (403, TO_DATE ('15-07-2025','DD-MM-YYYY'),42);


-- SQLite
INSERT INTO LesVilles VALUES ('Paris','France');
INSERT INTO LesVilles VALUES ('Londres','Angleterre');

INSERT INTO LesCircuits VALUES (403,'Paris','Paris',1160);

INSERT INTO LesEtapes VALUES (403,1,'Paris', 3);
INSERT INTO LesEtapes VALUES (403,2,'Londres', 2);

INSERT INTO LesProgrammations VALUES (403,'2025-03-04',30);
INSERT INTO LesProgrammations VALUES (403,'2025-07-15',42);


--Chat GPT
-- Insérer Paris et Londres dans LesVilles si elles n'existent pas déjà
INSERT OR IGNORE INTO LesVilles (nomV, pays) VALUES ('Paris', 'France');
INSERT OR IGNORE INTO LesVilles (nomV, pays) VALUES ('Londres', 'Royaume-Uni');

-- Insérer le circuit 403 dans LesCircuits
INSERT INTO LesCircuits (numC, vDep, vArr, prix)
VALUES (403, 'Paris', 'Paris', 1160);

-- Insérer les étapes du circuit 403 dans LesEtapes
INSERT INTO LesEtapes (numC, rang, vEtape, nbJours)
VALUES (403, 1, 'Paris', 3); -- Première étape : Paris, 3 jours
INSERT INTO LesEtapes (numC, rang, vEtape, nbJours)
VALUES (403, 2, 'Londres', 2); -- Deuxième étape : Londres, 2 jours

-- Insérer les programmations du circuit 403 dans LesProgrammations
INSERT INTO LesProgrammations (numC, dateDep, nbPlaces)
VALUES (403, '2025-03-04', 30); -- Programmation 1 : 4 Mars 2025, 30 places
INSERT INTO LesProgrammations (numC, dateDep, nbPlaces)
VALUES (403, '2025-07-15', 42); -- Programmation 2 : 15 Juillet 2025, 42 places
```